

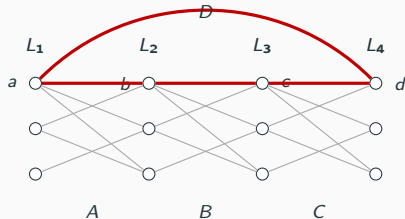
Counting 4-Cycles in Fully Dynamic Graphs

A Cyclic-Join Perspective

Speaker 1 Speaker 2 Speaker 3 Speaker 4

Algorithms seminar – mock PODS 2025

The problem, in one picture



A four-way **cyclic join** on a database:

$$J = A \bowtie B \bowtie C \bowtie D$$

```
- A(a,b), B(b,c), C(c,d), D(d,a)
SELECT COUNT(*)
FROM A, B, C, D
WHERE A.b = B.b
      AND B.c = C.c
      AND C.d = D.d
      AND D.a = A.a
```

One tuple in J = one 4-cycle in the layered graph.

So $|J|$ is a count of 4-cycles.

What “fully dynamic” actually demands

Edges arrive and leave one at a time. After every update the system must report the exact 4-cycle count.

Cost model: *worst-case* update time

The maximum work on *any single* update – not the average, not the amortised total.

Why worst-case, not amortised?

- Databases need **predictable per-update latency**.
- A 1-second hiccup once every 1000 updates is a production incident, even if the average is fast.
- Stronger guarantee $\Rightarrow$ harder problem.

The target: beat recompute-from-scratch with a per-update *worst-case* guarantee.

The landscape: three patterns on ≤ 4 vertices

Pattern	Best dynamic update time
Triangle (3-cycle)	$O(\sqrt{m})$
4-clique	$O(m)$
4-cycle	$O(m^{2/3})$

- m = number of edges in the host graph.
- That $2/3$ was the state of the art for over a decade.

Why is the 4-cycle bound stuck at $m^{2/3}$?

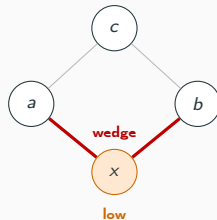
Algorithm of Hanauer, Henzinger, Hua (ICALP 2022):

- Split vertices into **low** (degree $\leq m^{1/3}$) and **high** (degree $> m^{1/3}$).
- Maintain *wedge matrix* $W[u, v] = \# \text{paths } u-x-v \text{ with } x \text{ low-degree}$.
- Edge update (u, v) : recompute $W[\cdot, v]$ for all low- x neighbours of u — at most $m^{1/3}$ of them, touching $m^{1/3}$ entries each $\Rightarrow m^{2/3}$ work.

This split is the **natural ceiling for combinatorial methods**.

Lower bound (OMv conjecture): $\Omega(m^{1/2-\gamma})$ – gap remains open.

Wedge through low-degree x



W	a	b	c
a	—	1	0
b	1	—	0
c	0	0	—

$W[a, b]=1$: one path $a-x-b$ via low x

The 2025 surprise (Assadi & Shah)

Theorem (Assadi–Shah, PODS 2025)

There is an algorithm maintaining the exact 4-cycle count in worst-case time

$$O(m^{2/3-\varepsilon}), \quad \varepsilon > 0$$

using **fast matrix multiplication** (FMM).

Why is this surprising?

- FMM is a **batch** primitive: multiply two big matrices at once.
- Incremental view maintenance is **streaming**: edges arrive one at a time.
- Folklore said: *FMM cannot give worst-case dynamic improvements.*

Their trick: *phases*. Precompute one matrix product across a batch of $m^{1-\delta}$ updates, while answering queries against the *previous* batch.

Previous approaches vs. Assadi–Shah

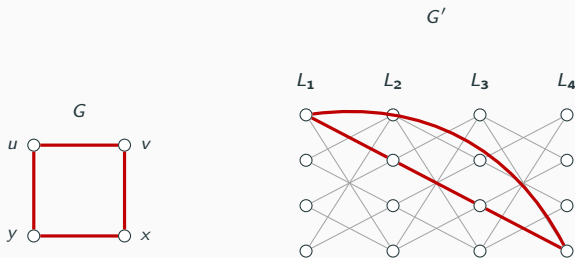
	Combinatorial (prior)	Assadi–Shah 2025
Worst-case update time	$O(m^{2/3})$	$O(m^{2/3-\varepsilon}), \varepsilon > 0$
Core technique	Wedge-matrix bookkeeping	Fast matrix mult. (FMM)
Unit of work	One entry per edge update	One product per <i>phase</i>
Key structure	$W[u, v] = \# \text{low-wedges}$	$A^{L*} \cdot B_{\ell, DD}$ (rectangular)
Beaten by FMM lower bd?	No (combinatorial ceiling)	Yes

Why FMM changes things:

- Combinatorial methods touch entries *one at a time*: cost = entries touched per update.
- FMM multiplies a whole matrix at once: cost is **sublinear in #entries** for rectangular shapes.
- **Phases** bridge streaming and batching: collect $\sim m^{1-\delta}$ updates, pay FMM once, spread the cost.

The cyclic-join lens

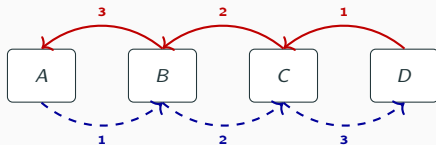
Assadi-Shah is dense. Our lens: **IVM for cyclic joins**.



Every general-graph edge $\rightarrow$ **4 layered-graph edges**, one per biadjacency matrix A, B, C, D . The architecture (degree classes, matrix products, where each step lives) becomes *visible*.

The update protocol

A single edge (u, v) in G becomes **four edges** in G' – one in each of A, B, C, D . **Order matters.**



insertion: $D \rightarrow C \rightarrow B \rightarrow A$

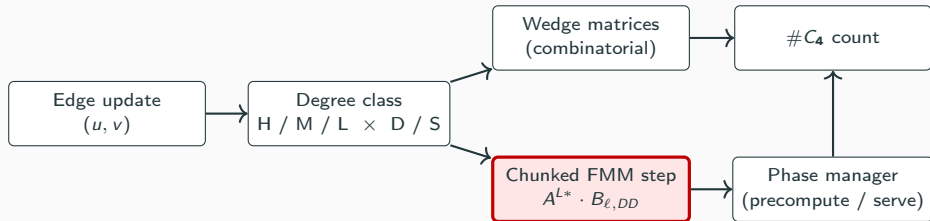
deletion: $A \rightarrow B \rightarrow C \rightarrow D$

Query window

Read the count *right after* the D -write. The edge is then **present in D , absent from A, B, C .**

This asymmetry forces every length-3 walk the algorithm counts to be a length-3 *path* (no vertex repeats). Without it, walks could revisit through the freshly written edge and the count would over-report.

Architecture at a glance



- **Degree classes** (H/M/L, Dense/Sparse) decide which routine handles a class pair.
- **Wedge matrices** handle the combinatorial class pairs incrementally – as in the classical $m^{2/3}$ algorithm.
- **One chunked FMM step** (red box) handles the Low-Dense class pair. This is where the speedup lives.
- **Phases** reconcile batched FMM with worst-case per-update time.

How FMM is used

Of the entire machinery, **one product** carries the speedup:

$$A^{L*} \cdot B_{\ell, DD} \quad (\text{rectangular, indexed by Low rows and Dense–Dense columns})$$

Two ideas reconcile batched FMM with worst-case per-update time:

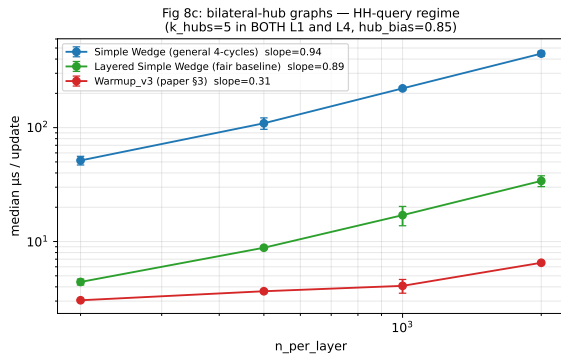
- **Chunking**: amortise one product across $\sqrt{m^{2/3-\varepsilon}}$ updates.
- **Phases**: while the current phase serves queries from the previous product, the next product is precomputed in the background – each update doing its $O(m^{2/3-\varepsilon})$ share.

Main bound (Assadi–Shah, PODS 2025)

Worst-case update time $O(m^{2/3-\varepsilon})$ for some constant $\varepsilon > 0$.

Our paper gives a closed-form certificate: $\varepsilon = 1/24$ at the conjectural $\omega = 2$. *See §7 for the constraint system and proof.*

Empirical check: where does the machinery fire?



Setup. Restricted variant (warm-up), NumPy GEMM in place of FMM. Baseline: layer-aware Simple-Wedge.

Testing the shape of the win, not asymptotics.

Finding: the machinery pays off on bilateral hubs – high-degree vertices on both sides of the cyclic ordering.

On simpler inputs (unilateral, sparse, no hubs), the wedge baseline already captures most of the benefit.

The routing machinery is not a generic accelerant – it locates a **specific structural regime** where FMM bandwidth pays for itself.

Takeaway

The surrounding machinery exists to feed one well-conditioned FMM step.

Two angles converged at that pressure point:

- **Algebraic**: in the constraint system, the FMM inequality is the one that binds.
- **Empirical**: among input regimes, the bilateral-hub one is the one where the machinery wins.

Open directions:

- Push the closed-form bound to the real $\omega \approx 2.371$.
- Other cyclic-join sizes: 5-cycles, 6-cycles, k -cycles.
- Adaptive routing: detect bilateral-hub regimes online, fall back otherwise.

More broadly: **IVM for cyclic joins** is a clean lens on the FMM-vs-streaming tension.

Thank you.

Questions?